

情報工学科 講義 デジタル信号処理B (2025/06/26)

第4回 (Aからカウントして第11回) 微分可能信号処理

情報工学科 准教授 高道 慎之介



Takamichi Laboratory
慶應義塾大学 高道研究室

ディジタル信号処理Bの授業予定 (仮)

第XX回	日付	内容 (順次変わっていくので予想)	
第01回	2025/05/29	適応信号処理, 信号処理	昨年のBの途中まで
第02回	2025/06/12		
第03回	2025/06/19	ベクトル・行列微分の基礎と応用	新規内容
第04回	2025/06/26	微分可能信号処理	
第05回	2025/07/03		
第06回	2025/07/10		
第07回	2025/07/17	総合演習. 期末試験の練習としての立ち位置.	
期末試験	2025/07/??	(日程は後日アナウンス)	

前回の演習課題

演習

関数 $f(\mathbf{X})$	微分 $\frac{\partial f}{\partial \mathbf{X}}$
$\text{tr}(\mathbf{A}^\top \mathbf{X})$	$\mathbf{A}$ (1) これを証明せよ
$\text{tr}(\mathbf{A}\mathbf{X})$	$\mathbf{A}^\top$
$\text{tr}(\mathbf{X}\mathbf{A}\mathbf{X}^\top)$	$\mathbf{X}\mathbf{A}^\top + \mathbf{X}\mathbf{A}$
$\text{tr}(\mathbf{X}\mathbf{A}\mathbf{X}^\top)$ ($\mathbf{A}$ が対称行列)	$2\mathbf{X}\mathbf{A}$ (2) これを証明せよ
$(\mathbf{X}\mathbf{w} - \mathbf{a})^\top (\mathbf{X}\mathbf{w} - \mathbf{a})$	$2(\mathbf{X}\mathbf{w} - \mathbf{a})\mathbf{w}^\top$

解説

1. 省略。 $\text{tr}(\mathbf{A}^T \mathbf{X})$ の場合と同じようにやれば解ける。
2. -
 - a. $\text{tr}()$ をつけ括弧を展開する。
 - b. 以下の公式を利用する。
 - i. $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$ ($\mathbf{A}, \mathbf{B}$ は正方行列)
 - ii. $\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}^T)$ ($\mathbf{A}$ は正方行列)
 - iii. $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ ($\mathbf{A}, \mathbf{B}$ は正方行列でなくてもよい)
 - c. $\text{tr}(\mathbf{A}^T \mathbf{X})$ と $\text{tr}(\mathbf{XAX}^T)$ の形になるよう変形する。

本日の内容

音のAIはすごい！ (Google NotebookLM に前回の講義資料を入れてみた)

 Vector and Matrix Calculus in Digital Signal Processing

新機能！一般公開で共有する

共有

設定

ソース

+ 追加

🔍 検索

すべてのソースを選択 ☒

 10.pdf

☒



https://drive.google.com/file/d/1fsVV9X8KO9t12uV1nU4Xy2hAV0IRApK7/view?usp=drive_link

チャット



Vector and Matrix Calculus in Digital Signal Processing

1 ソース

この講義資料は、デジタル信号処理Bの授業内容を詳述しており、特にベクトルと行列の微分の基礎と応用を扱っています。資料は、ノイズキャンセリングの復習から始まり、スカラー関数をスカラ、ベクトル、行列で微分する方法を段階的に解説しています。さらに、決定論的信号処理と統計的信号処理の違いにも触れ、多変量正規分布などの概念を紹介しています。最終的には、ベクトル値関数をベクトルで微分するヤコビ行列についても説明し、演習問題や公式の証明を通じて理解を深めることを目的としています。

📌 メモに保存



 メモを追加

 音声概要

 マインドマップ

入力を開始します...

1 個のソース



ベクトル・行列微分がデジタル信号処理のどのような側面に適用され、その応用がと



Studio

音声解説

 音声解説をより多くの言語で作成しましょう。詳細

 会話の生成元: 1 ソース...

これには数分かかることがあります。その他の操作ができます。

メモ

+ メモを追加

 学習ガイド

 ブリーフィング・ドキュメント

 よくある質問

 タイムライン



保存したメモがここに表示されます
チャットメッセージを保存して新しいメモを作成するが、上の [メモを追加] をクリックします。

NotebookLM は不正確な場合があります。回答は確認してください。

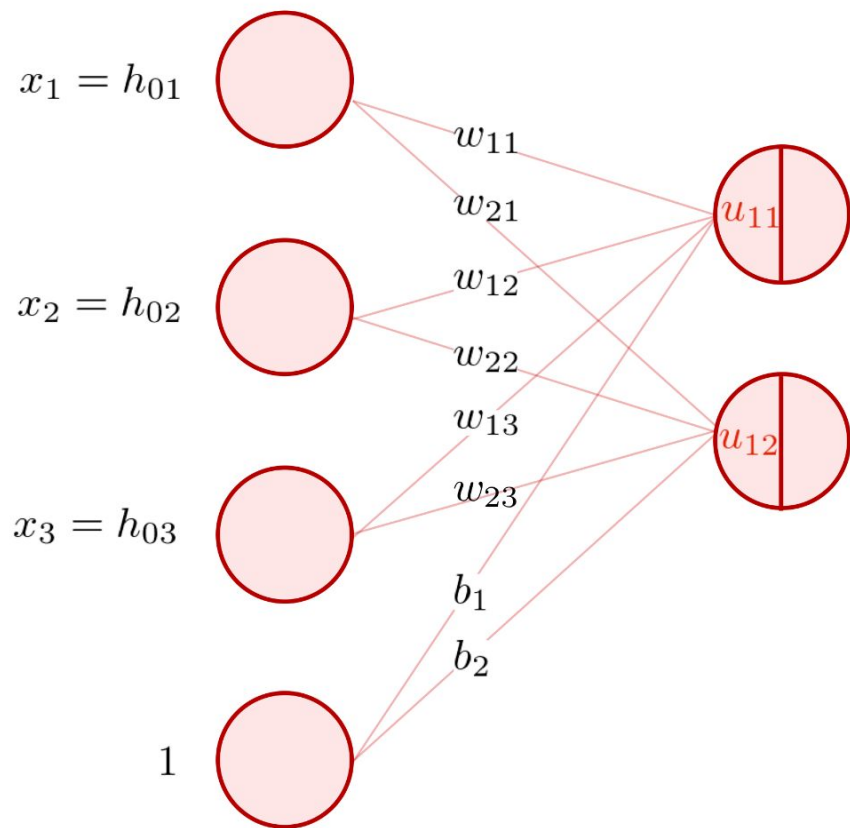
本日の内容

- 深層学習と信号処理を接続しよう！両者は別々のものではない

微分可能デジタル信号処理 (differentiable digital signal processing; DDSP) と呼ばれる処理を理解しよう！

深層學習

全結合型ニューラルネットワーク (FFNN)

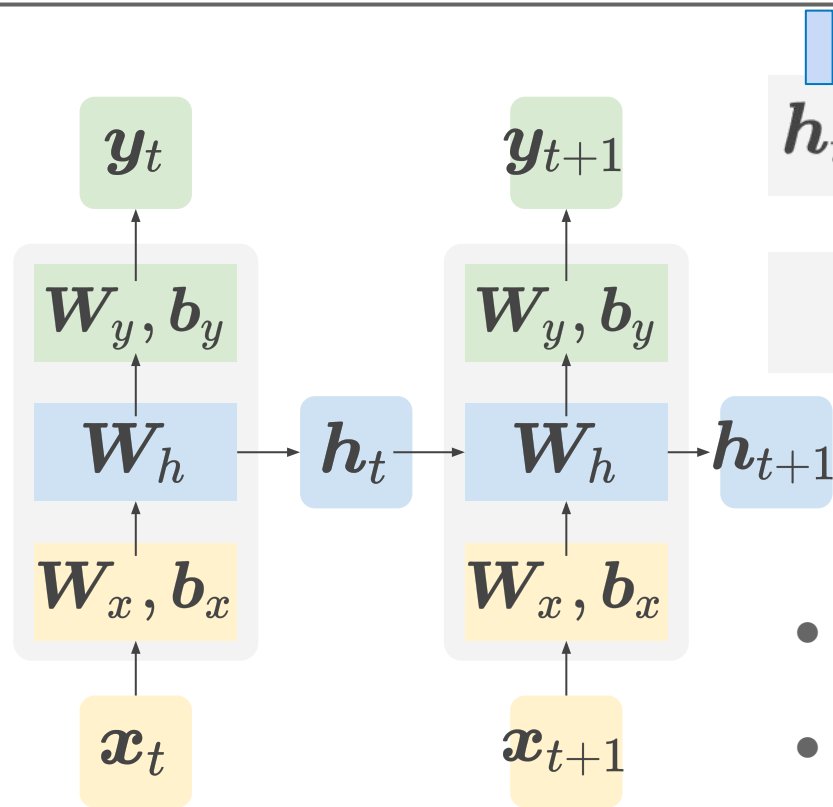


$$y = Wx + b$$



- 行列の式で書くことができる
- バイアスは“1”の入力と見做せる
- この線形変換の後に活性化関数 (たとえば sigmoid, tanh 関数) を作用させることで、非線形変換を実現できる

リカレントニューラルネットワーク (RNN)



$$h_t = f_h (W_h h_{t-1} + W_x x_t + b_x)$$

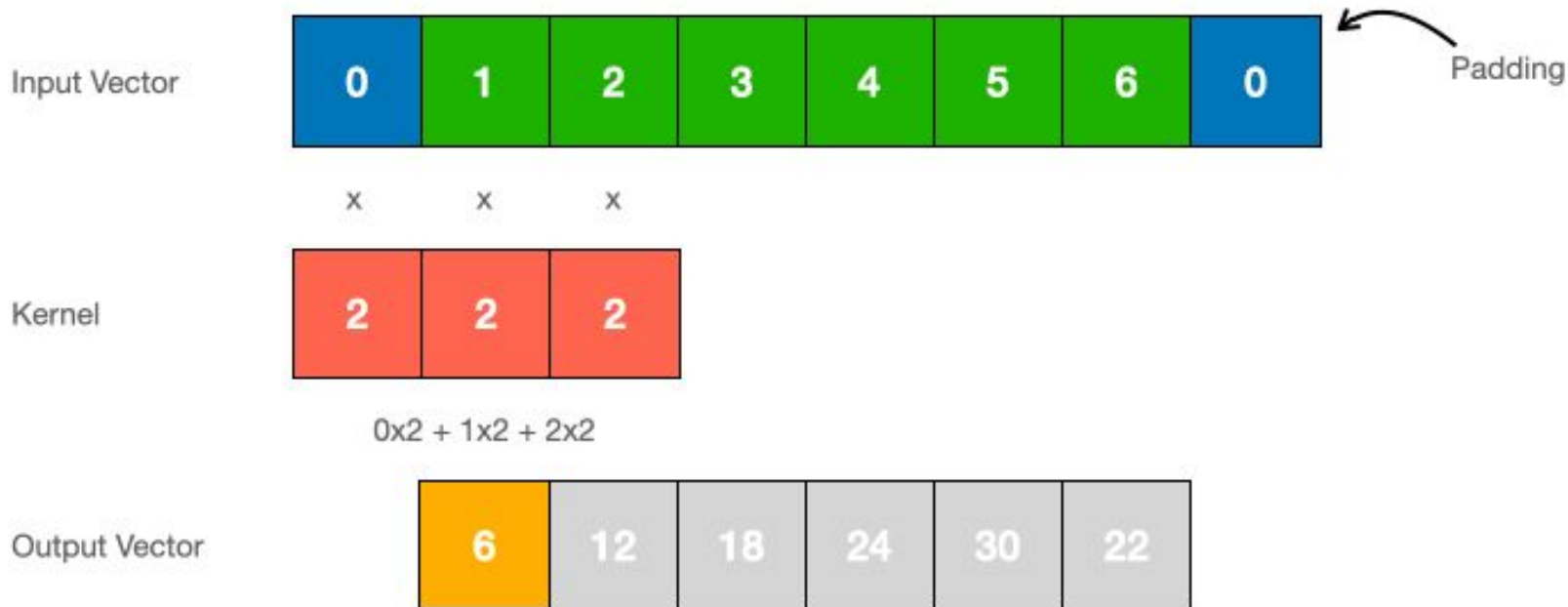
$$y_t = f_y (W_y h_t + b_y)$$

ベクトル要素ごとの
非線形変換

- 1つ前の時刻の内部状態 h を使う
- 入力 x に作用する項、1つ前の時刻の内部状態 h に作用する項、出力 y を計算する項

(実際には活性化関数 f_h, f_y もあるので注意)

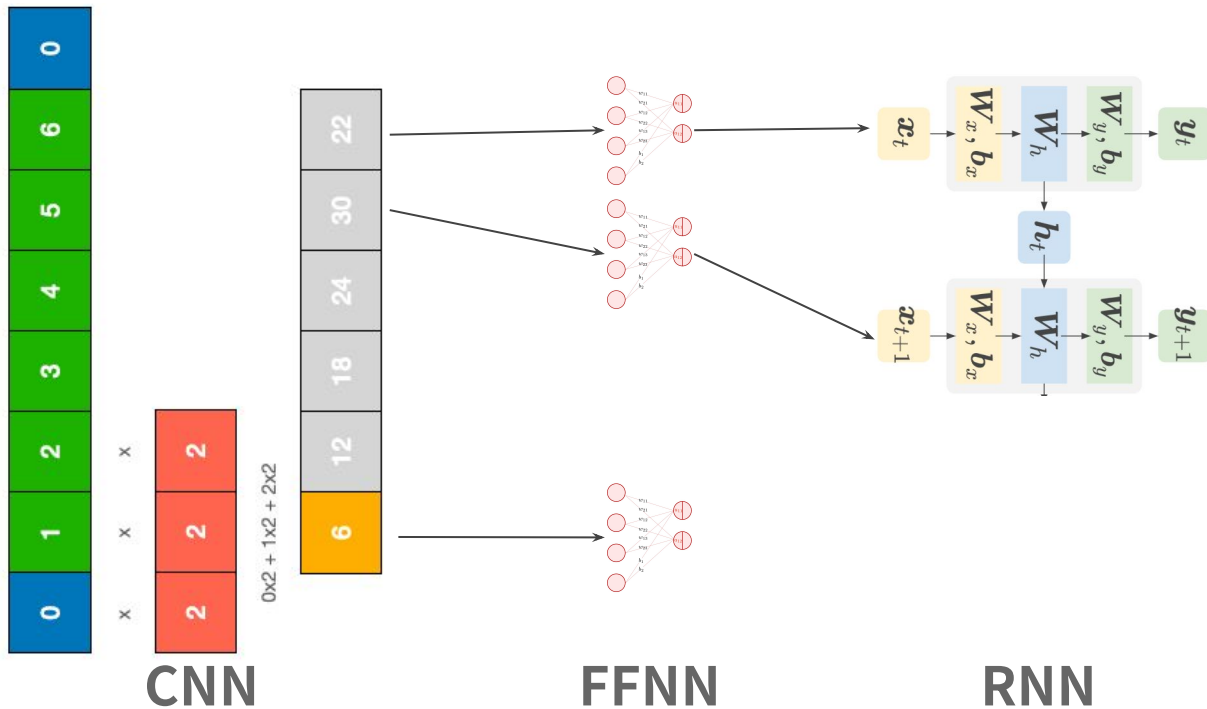
畳み込みニューラルネットワーク (CNN)



$$y = h \star x$$

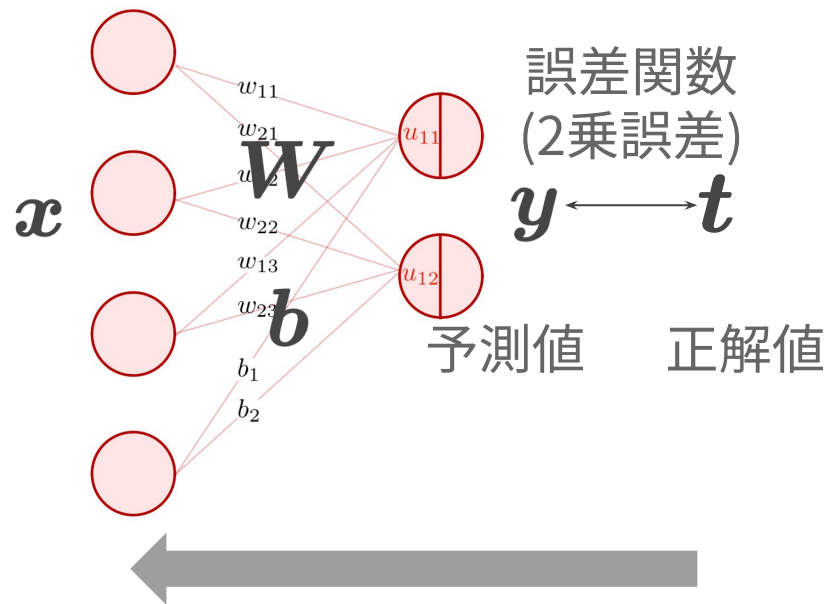
実は、これまで習った畳み込み演算とは違う (後述)

複数のニューラルネットワークを結合できる



バックプロパゲーション

$$y = Wx + b$$



誤差関数のベクトル表現

$$\text{error} = \sum_i (y_i - t_i)^2 = (\mathbf{y} - \mathbf{t})^\top (\mathbf{y} - \mathbf{t})$$

勾配法による更新 (α は学習係数)

$$W \leftarrow W - \alpha \frac{\partial \text{error}}{\partial W}$$

前回にやった、“スカラー関数を行列で微分”

実は，今まで習ったデジタル信号処理は
ニューラルネットワークに似た式で記述可能

(復習) 離散フーリエ変換とその逆変換

- 離散 Fourier 変換：離散時間信号 $\rightarrow$ 離散周波数

$$X[k] = \sum_{n=0}^{N-1} x[n] \exp\left(-j\frac{2\pi kn}{N}\right) \quad \text{DFT}[x[n]] = X[k]$$

$(k \in \{0, 1, \dots, N-1\})$

- 逆離散 Fourier 変換：離散周波数 $\rightarrow$ 離散時間信号

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \exp\left(j\frac{2\pi kn}{N}\right) \quad \text{DFT}^{-1}[X[k]] = x[n]$$

$(n \in \{0, 1, \dots, N-1\})$

その行列表現 (ディジタル信号処理 A の課題)

$$\mathbf{X} = \mathbf{W}_F \mathbf{x}$$

$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{array}{c} \boxed{\text{N-by-N} \\ \text{matrix } W} \end{array} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

```
import numpy as np

def DFT_matrix(N):
    W = np.zeros((N,N), dtype=complex)
    """
    Generate DFT matrix of size N.
    """
    for n in range(N):
        for k in range(N):
            W[n][k] = np.exp(- 2j * np.pi * n * k / N)
    return W
```

全結合型ニューラルネットワークと同じ式の形！
(要素は複素数を取り、バイアス項が無いという違いはある)

振幅と位相の計算は活性化関数のように見做せる

振幅 (ベクトル要素ごとの絶対値)

$$f(a + jb) = \sqrt{(a^2 + b^2)}$$

$$f(a + jb) = \tan^{-1} \frac{b}{a}$$

位相 (ベクトル要素ごとのtanh)

$$\mathbf{X} = \mathbf{W}_{\mathcal{F}} \mathbf{x}$$

$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} \text{N-by-N} \\ \text{matrix } W \end{bmatrix} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

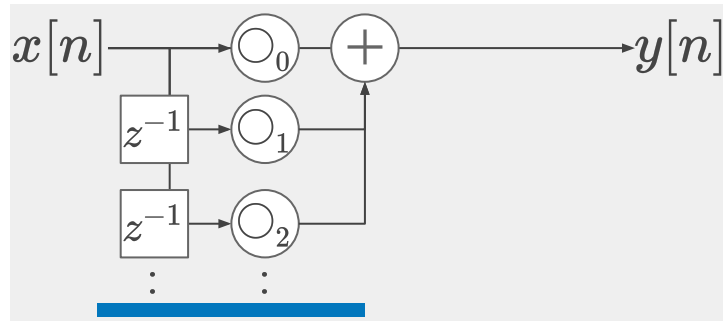
全結合型ニューラルネットワークにおける 線形変換 → 活性化関数と同じような計算！つまりバックプロパゲーションできる！

(復習) FIR フィルタ

差分方程式 入力にのみ依存

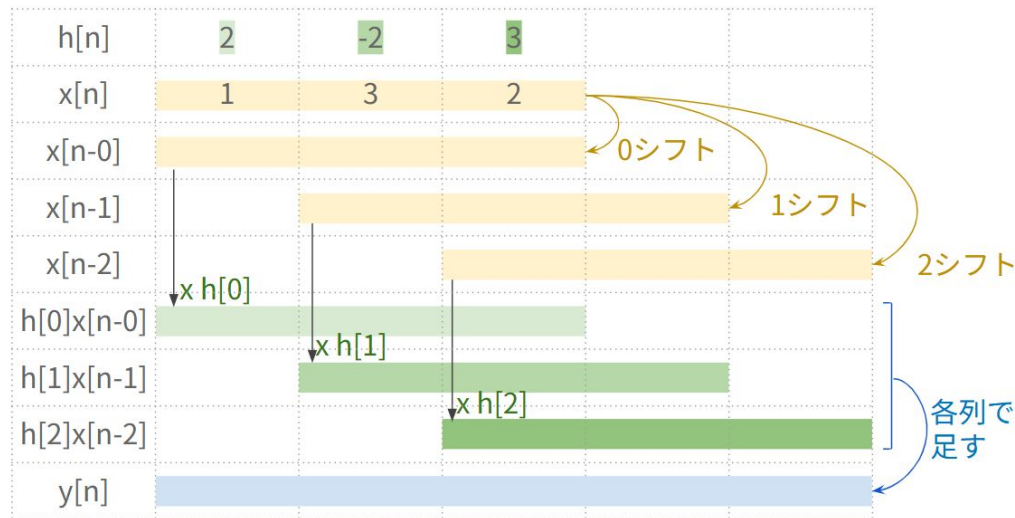
$$y[n] = \sum_m \bigcirc_m x[n - m]$$

ブロック図



フィードフォワードのみ

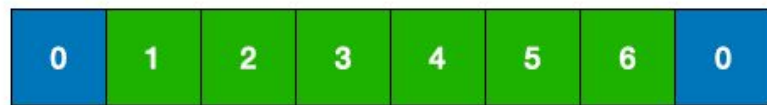
要は畳み込み演算



“片方を固定した状態でもう片方をずらしながら加算する”
→ 畳み込みニューラルネットワークの演算と似ているっぽい

注意：畳み込みニューラルネットワークの演算は畳み込み演算ではない！

畳み込みニューラルネットワーク

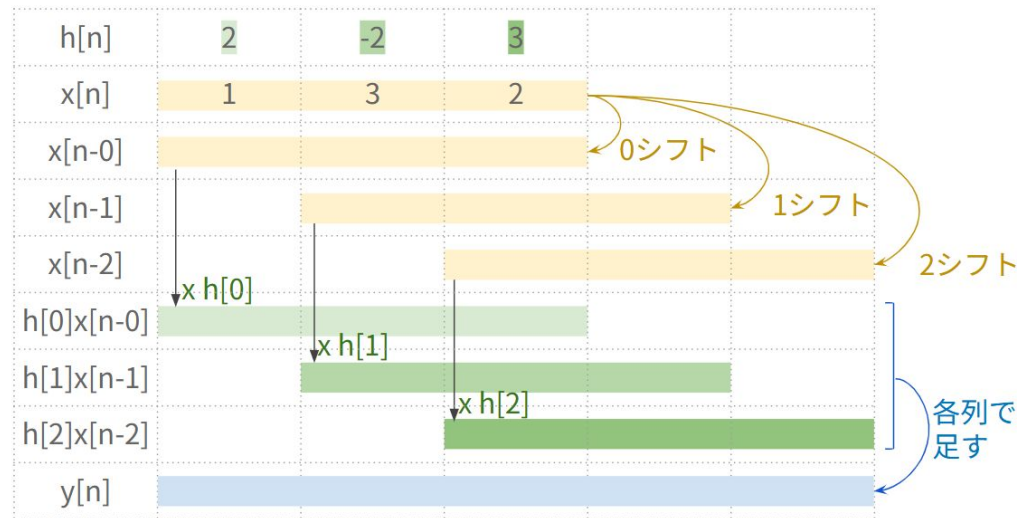


x x x

$$0 \times 2 + 1 \times 2 + 2 \times 2$$

$$y = h \star x$$

FIRフィルタ



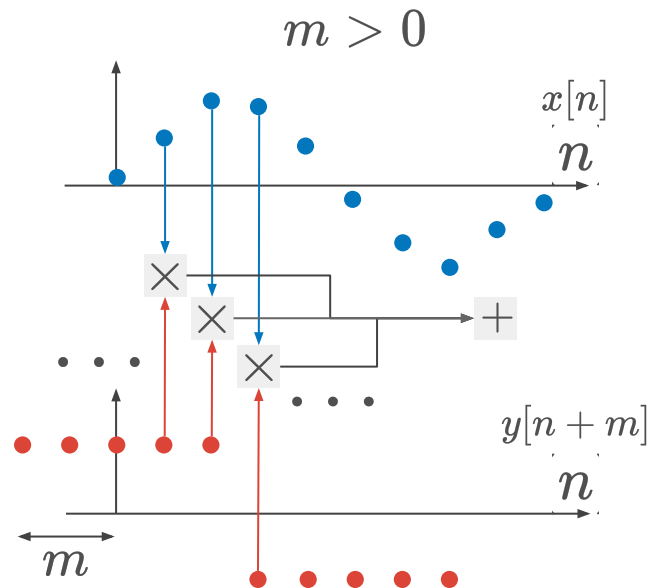
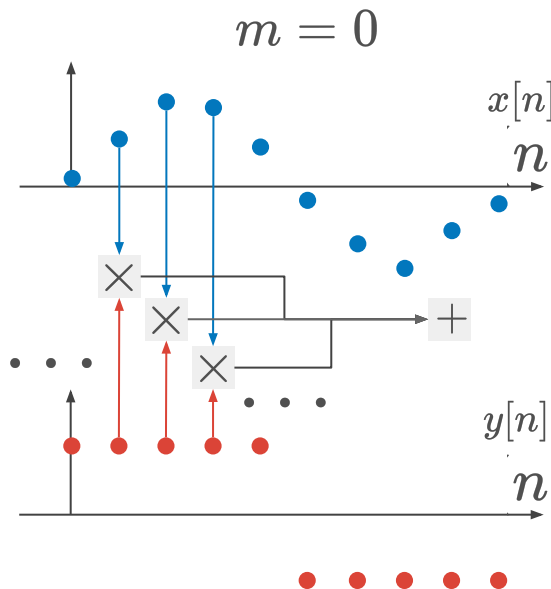
$$y = h * x$$

時間をずらすことは共通しているが、カーネル (フィルタ) のかけ方が違う
→ **実は、畳み込みニューラルネットワークのほうは相互相関関数！**

(復習) 相互相関関数

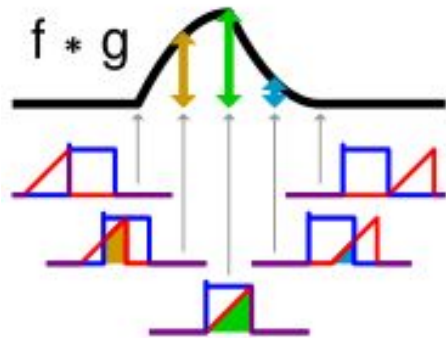
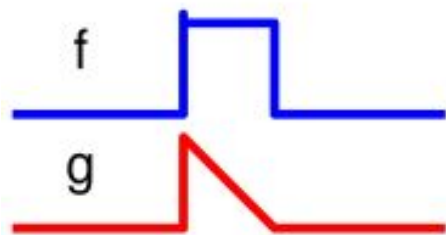
$$\phi_{xy}[m] = \frac{1}{N} \sum_{n=0}^{N-1} x[n]y[n+m]$$

$m \leq 0$ と考えれば前のページと等価

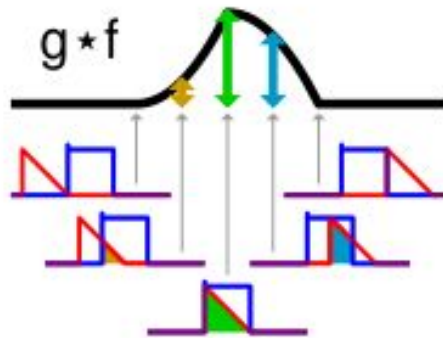
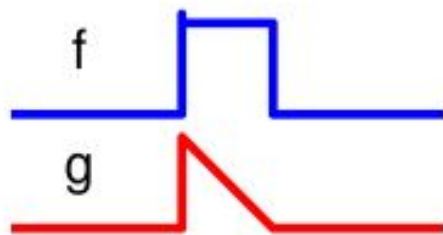


時間反転したフィルタの畳み込みは相互相関と等価

FIRフィルタの
“畳み込み”
Convolution



CNNの
“畳み込み”
Cross-correlation



FIRフィルタのフィルタ (g) を時間反転して相互相関を計算すると、CNNの畳み込み結果に一致する。

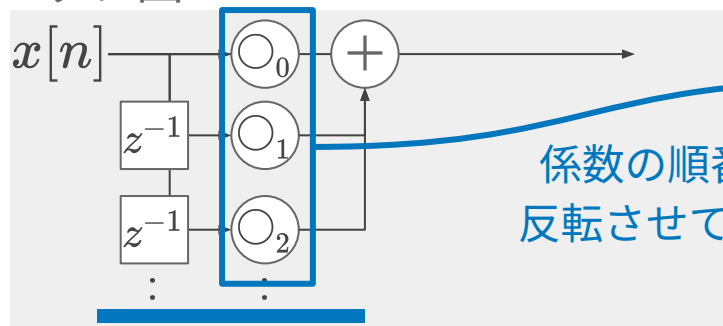
講義では説明しないが、畳み込みと相互相関の定義式から証明可能。
 $h'(t) = h(-t)$ を導入することで可能。

FIR フィルタのニューラルネットワーク的表現

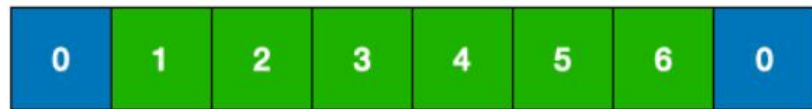
差分方程式 入力にのみ依存

$$y[n] = \sum_m \circ_m x[n - m]$$

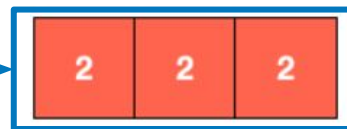
ブロック図



フィードフォワードのみ



x x x



$0x2 + 1x2 + 2x2$



係数の順番を
反転させて変換

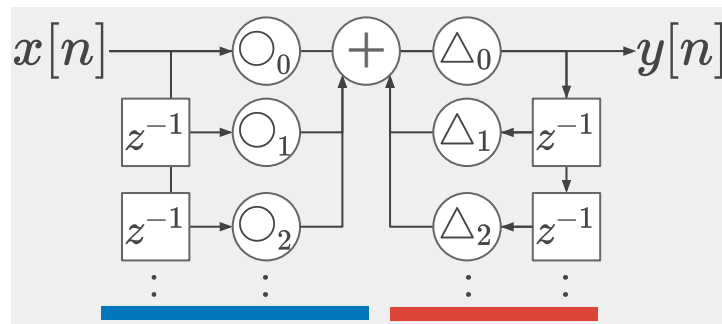
(復習) IIR フィルタ

差分方程式

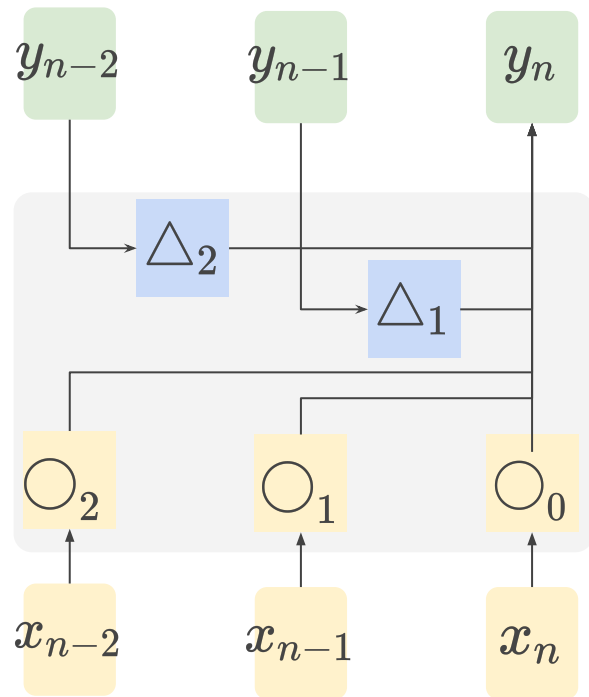
過去の出力にも依存

$$y[n] = \sum_m \bigcirc_m x[n - m] + \sum_l \triangle_l y[n - l]$$

ブロック図

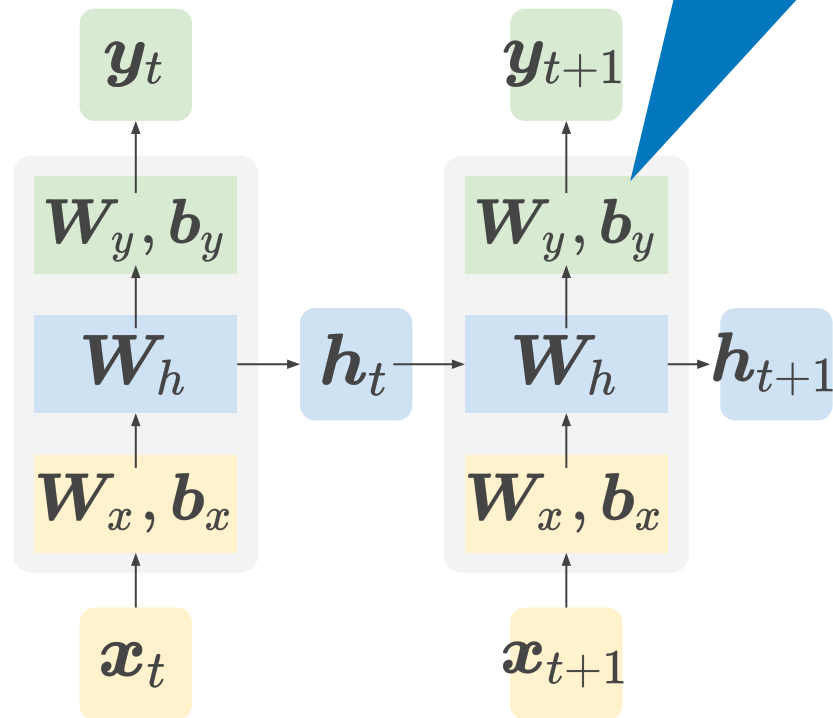


フィードバック構造あり

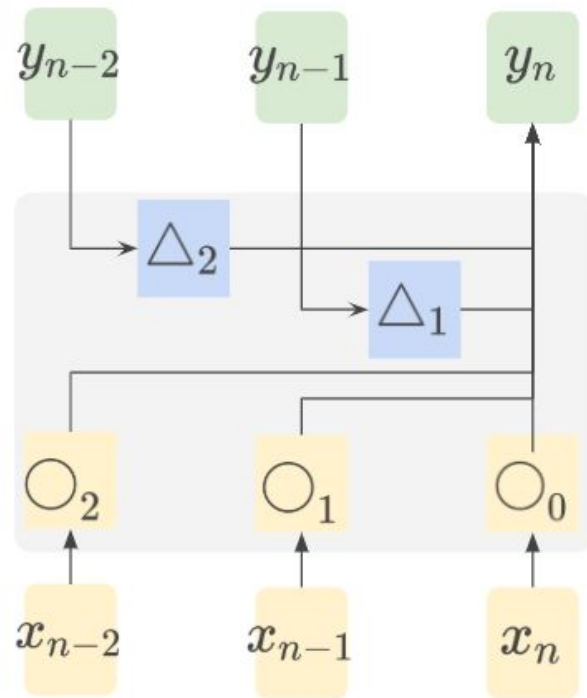


なんか似てるよね？

出力層が恒等写像なら
より似る (等価ではない)



RNN



IIRフィルタ

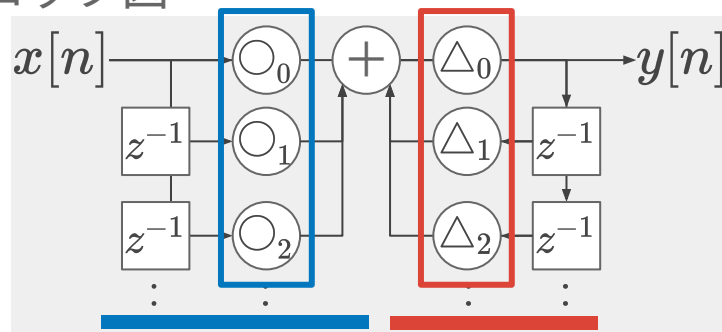
そのニューラルネットワーク的表現

差分方程式

過去の実出力にも依存

$$y[n] = \sum_m \bigcirc_m x[n-m] + \sum_l \triangle_l y[n-l]$$

ブロック図



フィードバック構造あり

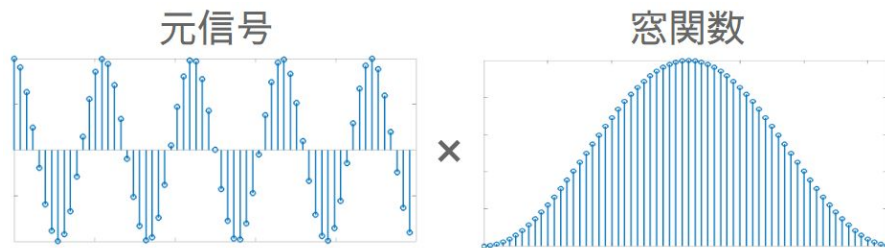
CNNを使って記述するなら、

- フィードフォワードは入力に対する畳み込み層
- フィードバックは過去の実出力に対する畳み込み層

で記述可能．フィルタ係数の順番は逆転させる．

(復習) 窓関数の役割

- 窓関数によるリップルの抑圧
 - 信号長を対象に合わせて変更することは通常できないため、両端のつながりの影響を少なくする窓関数を信号に乗算してリップルを抑制する。



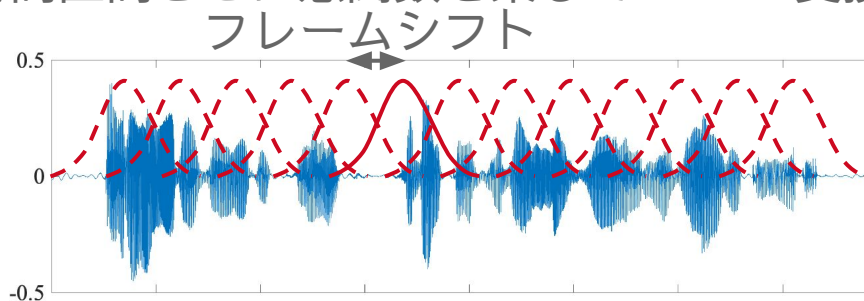
- 離散窓 Fourier 変換（代表例は次のスライド）
 - 離散時間信号 $x[n]$ に対し, 信号区間の中心から離れるほど減衰する重み関数 (窓関数) $w[n]$ を乗じて離散 Fourier 変換

$$X[k] = \sum_{n=0}^{N-1} w[n] x[n] \exp\left(-j \frac{2\pi kn}{N}\right)$$

窓関数の乗算

(復習) 短時間 Fourier 変換： 時間変化する信号に対する分析

- 時々刻々と変化する時系列信号を考える。
 - 長い時間区間の信号では、スペクトルの時間変化に意味がある場合が多い
 - (時間区間全体のスペクトルはあまり意味を持たない)
- 短時間 Fourier 変換 (short-time Fourier transform: STFT)
 - 短時間の時間区間ごとに窓関数を乗じて Fourier 変換

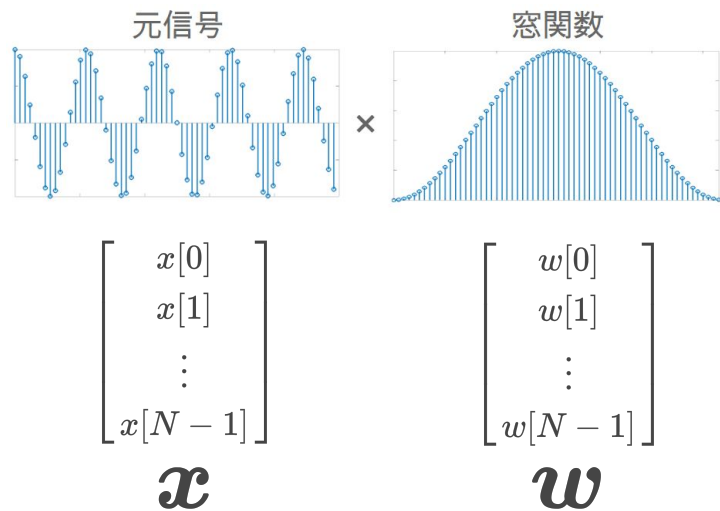


- 連続系での数式表現 (離散の場合も基本的に同じ)

$$X(\omega, t) = \int_{-\infty}^{\infty} x(\tau) \underbrace{w(\tau - t)}_{\text{時刻 } t \text{ を中心とした窓関数}} \exp(-j\omega\tau) d\tau$$

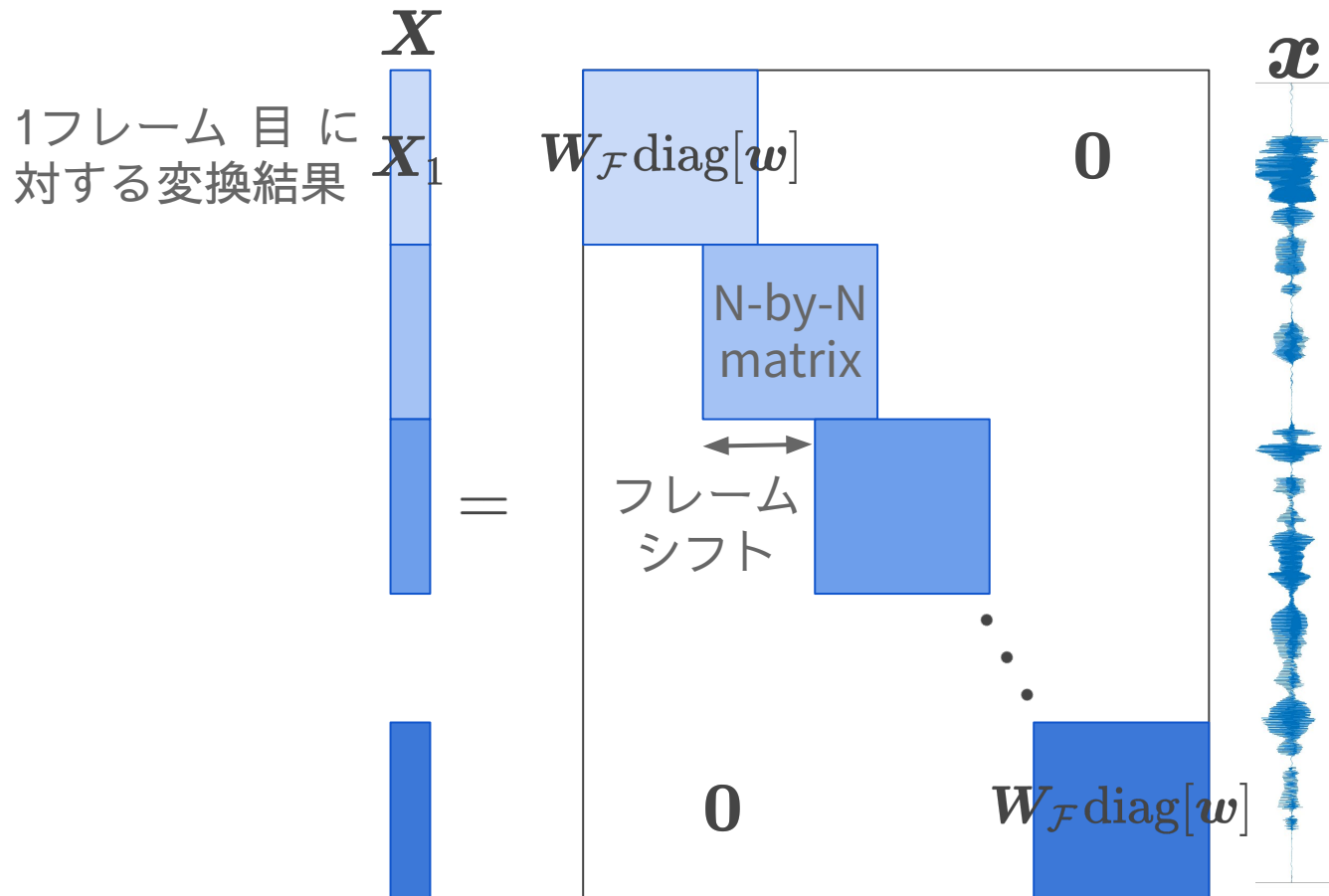
時刻 t を中心とした窓関数

窓かけはアダマール積 (要素積)

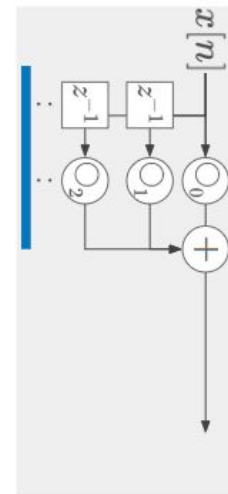
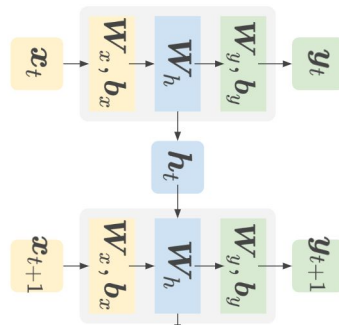
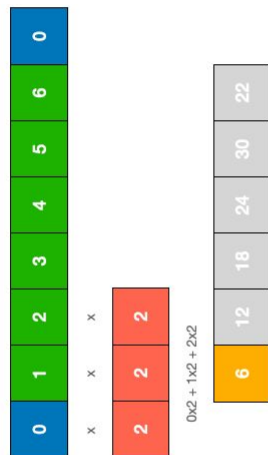
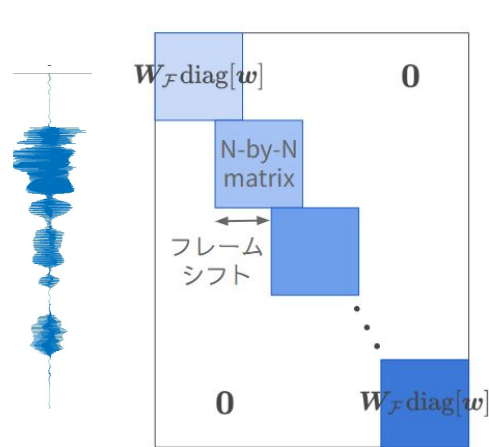


$$\boldsymbol{x} \odot \boldsymbol{w} = \text{diag}[\boldsymbol{w}] \cdot \boldsymbol{x} = \begin{bmatrix} x[0]w[0] \\ x[1]w[1] \\ \vdots \\ x[N-1]w[N-1] \end{bmatrix}$$

短時間フーリエ変換も行列で記述できる



ニューラルネットワークと結合もできる (もちろんバックプロパゲーションもできる)



誤差関数
(2乗誤差)

$y \longleftrightarrow t$

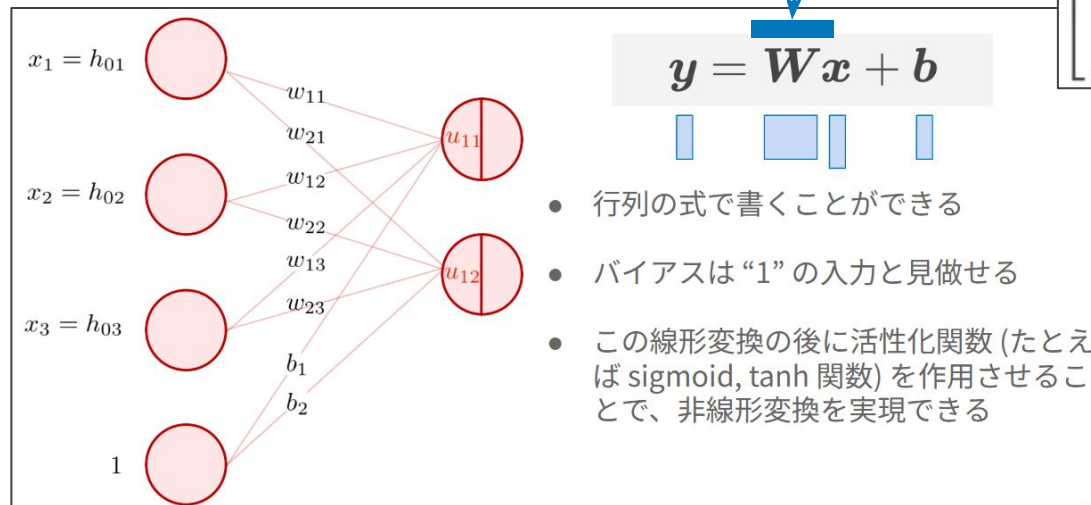
予測 正解

FIR

バックプロパゲーション

信号処理の変換を初期値にすることもできる

フーリエ変換行列で初期化して
バックプロパゲーションで更新
→ 最初はフーリエ変換の特徴を使い、
さらに良い特徴を学習で求める



$$X = W_F x$$
$$\begin{bmatrix} X[0] \\ \vdots \\ X[k] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{matrix} \text{N-by-N} \\ \text{matrix } W \end{matrix} \begin{bmatrix} x[0] \\ \vdots \\ x[n] \\ \vdots \\ x[N-1] \end{bmatrix}$$

プログラミングしてみよう

jax/flax というライブラリを使います
<https://github.com/google/flax>



Flax: A neural network library and ecosystem for JAX designed for flexibility

jax / flax 概説

https://zenn.dev/turing_motors/articles/4a7e80c4a47b65

JAXは非常にざっくりいうと、**機械学習用に設計されたNumPy**と説明することができます。実際、JAXの数値関数用のAPIは、NumPyと高度に互換性があり、NumPyと基本的には全く同じようにコードを実装することが可能です。このNumPy APIに加えて、機械学習に役立つ拡張として、次のような特徴があります。

JAXの使い方を紹介してきましたが、実はこのままでは深層学習の処理を直接実行すること簡単ではありません。そのために、**JAXをベースとした機械学習ライブラリを組み合わせる**必要があります。JAXを基盤とした機械学習ライブラリは複数ありますが、**Flax**はその中でも最も人気があるフレームワークの一つです。

演習：ベース音に掛けられたエフェクトの パラメータを求めよう

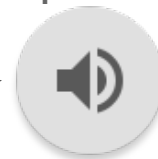
sample.wav



なにかのエフェクト



output.wav

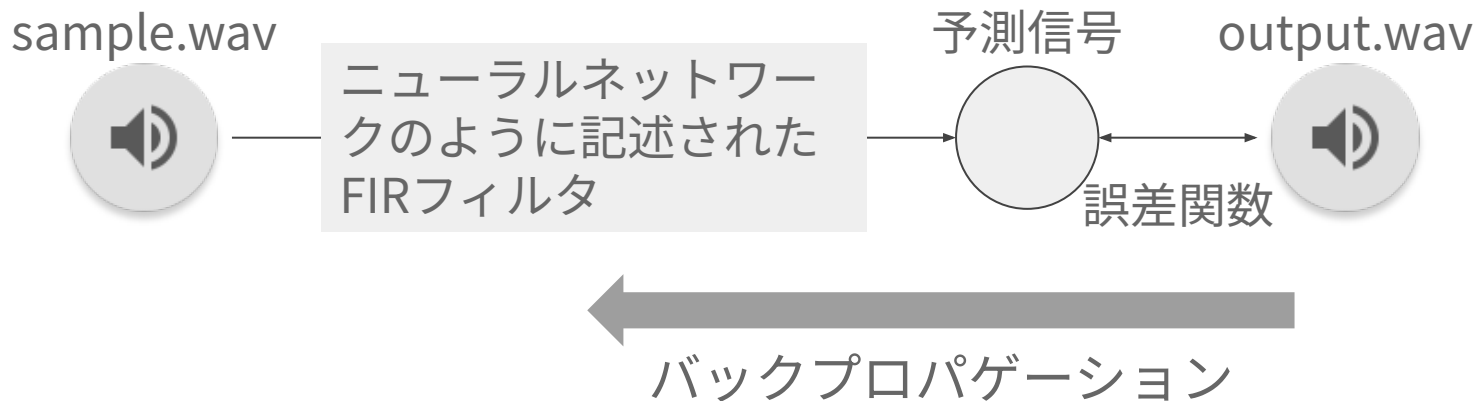


https://drive.google.com/file/d/1TqNuO3EJa3WxGkvhjizNko9T_QTSIzZg/view?usp=drive_link

元音源は https://maou.audio/se_inst_bass10/

<https://drive.google.com/file/d/1JE-GgUpZVxqY5SvY0nOOONDdwk6LTYAU/view?usp=sharing>

やること



Google colab で説明します

https://colab.research.google.com/drive/1sw1ykK_6lXHjudxF-N4pZvBClwOllik7?usp=sharing

自分で動かしたいときは，自分のドライブにコピーして，前のページの音ファイルをアップロードすること．

まとめ
(matome)

まとめ

- 深層学習
- デジタル信号処理を深層学習のように考える
- プログラミングで考える

課題 (exercise)

課題：どれか1つを実施せよ．いずれも点数は同じ．

1. 簡単コース

- a. google colab の例を参考に，別の FIR フィルタを設計しエフェクトをかけよ．そのフィルタを学習で推定できるかを確認し，その学習や収束の様子を考察せよ．

2. 普通コース

- a. IIR フィルタを設計しエフェクトをかけよ．以下，同上．

3. むずいコース

- a. <https://intro2ddsp.github.io/intro.html> を参考に，微分可能信号処理システムを構築せよ．以下，同上．